COMP3020: Machine Learning

Assignment 02

2026-11-19

Instructions

- Submit your assignment via Canvas.
- Your submission should include the zip file only (code + written part).
- You may use LaTeX or Word to create the written part of your assignment, but you must submit it in PDF format.

#	Section	Points
1	Basic Perceptron	10
2	Linear Separability with Feature Maps	15
3	Is the New Model Really Better?	10
4	Approximation vs. Estimation Error	10
5	Bias-Variance Tradeoff in Linear Models	38
6	Programming	17
	Total	**100**

1 Basic Perceptron (10 points)

Consider the following four training points in $\mathbb{R}^2$, presented in the order A, B, C, D:

Point	$\mathbf{x}$	y
A	$(2, 1)$	$+1$
B	$(0, 2)$	$+1$
C	$(-1, 0)$	-1
D	$(1, -2)$	-1

Exercise 1a (3). (*Tracing the perceptron updates.*) Run the standard perceptron algorithm starting from

$$\mathbf{w} = (0, 0), \qquad b = 0.$$

Cycle through the training examples in the order A, B, C, D until one full epoch is completed with no updates. For every example processed, record the activation

$$a = \mathbf{w}^\top \mathbf{x} + b,$$

whether an update occurs, and the resulting values of $(\mathbf{w}, b)$.

Exercise 1b (3). (*Decision boundary and geometric distance.*) Write the final decision boundary as an equation and sketch it together with the four training points. Which training point is closest to the decision boundary? Compute its geometric distance to the boundary.

Exercise 1c (2). (*Comparing margins of separating hyperplanes.*) Give a different separating hyperplane whose smallest geometric distance to the training points is strictly larger than that of the hyperplane obtained in Exercise 1b. What does this example tell you about which separating hyperplane the perceptron algorithm returns?

Exercise 1d (2). (*Effect of example ordering.*) Rerun the perceptron algorithm from

$$\mathbf{w} = (0, 0), \qquad b = 0,$$

but process the examples in the order D, C, B, A. Do you obtain the same separating hyperplane? Do you obtain the same number of updates? Relate your answer to the lecture's advice that the order of the training examples matters.

2 Linear Separability with Feature Maps (15 points)

Consider the four points

$$\mathbf{x}^{(1)} = (1, 1), \quad \mathbf{x}^{(2)} = (-1, -1)$$

with label $+1$, and

$$\mathbf{x}^{(3)} = (1, -1), \quad \mathbf{x}^{(4)} = (-1, 1)$$

with label -1.

Exercise 2a (5). (*Non-separability of XOR-type data.*) Prove that there is no pair $(\mathbf{w}, b)$, with $\mathbf{w} \in \mathbb{R}^2$ and $b \in \mathbb{R}$, that linearly separates these four points.

Hint: add the inequalities corresponding to the two positive points, and then add the inequalities corresponding to the two negative points.

Exercise 2b (5). (*Separating with a feature map and computing the margin.*) Consider the feature map

$$\phi(\mathbf{x}) = \begin{pmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{pmatrix}, \qquad \text{where } \mathbf{x} = (x_1, x_2).$$

Give a unit-norm vector $\mathbf{w}^\star \in \mathbb{R}^3$, with $b = 0$, that separates the mapped data. Compute the margin

$$\gamma = \min_i \, y^{(i)} (\mathbf{w}^\star)^\top \phi(\mathbf{x}^{(i)}).$$

Exercise 2c (3). (*Perceptron mistake bound.*) Using the perceptron convergence theorem, give an upper bound on the number of mistakes made by the perceptron on the mapped data.

Exercise 2d (2). (*Separability after adding a point.*) Now add the point

$$\mathbf{x}^{(5)} = \left(\frac{1}{2}, \frac{1}{2} \right)$$

with label -1. Is the mapped data still linearly separable through the origin?

If so, give a separator and a valid (not necessarily tight) upper bound on the number of perceptron mistakes. If not, prove that no such separator exists.

3 Is the New Model Really Better? (10 points)

Two models were each evaluated in 5 independent runs. In every run, each model was trained and tested on its own fresh random split; the splits are *not* shared between the two models, so the two sets of results are independent samples:

Run	Model X	Model Y
1	0.81	0.84
2	0.83	0.85
3	0.82	0.83
4	0.80	0.86
5	0.84	0.87

Exercise 3a (2). (*Mean and sample standard deviation.*) Compute the mean and sample standard deviation for each model.

Exercise 3b (2). (*Pooled two-sample t-statistic.*) Using a pooled two-sample t-test, compute the t-statistic for the difference between Model X and Model Y. Also state the corresponding degrees of freedom.

Exercise 3c (2). (*Significance at different levels and one-tailed testing.*) Determine whether the difference between the two models is statistically significant at each of the following levels:

$$\text{two-tailed } \alpha = 0.05, \qquad \text{two-tailed } \alpha = 0.01.$$

Also determine whether the claim

"Model Y is better than Model X"

is supported using a one-tailed test at $\alpha = 0.01$.

Exercise 3d (2). (*Effect of the number of runs.*) Suppose that only 3 runs per model had been performed, but the observed means and sample standard deviations were the same as those computed in Exercise 3a. Recompute the pooled two-sample t-statistic and determine whether the difference is significant at the two-tailed level

$$\alpha = 0.05.$$

What does this result tell you about reporting model performance based on only a very small number of runs?

Exercise 3e (2). (*Multiple comparisons.*) Suppose you propose Model Y and compare it independently against 3 different baseline models, using significance level

$$\alpha = 0.05$$

for each comparison. If Model Y is truly no better than any of the three baselines, what is the probability of obtaining at least one statistically significant "win" purely by chance?

4 Approximation vs. Estimation Error (10 points)

Let

$$x \sim \text{Uniform}[-2, 2].$$

The true label is

$$y = \begin{cases} +1, & |x| > 1, \\ -1, & |x| \le 1. \end{cases}$$

There is no noise in the labels. The error of a classifier is the probability that it predicts the wrong label.

Exercise 4a (2). (*Best threshold classifier.*) Let $\mathcal{F}_1$ be the set of all one-dimensional threshold classifiers of the form

$$\text{sign}(wx + b),$$

including classifiers that always predict the same label.

What is the smallest possible error of a classifier in $\mathcal{F}_1$? Give one classifier that achieves this error.

Exercise 4b (2). (*Best classifier with quadratic features.*) Now use the two features

$$x \qquad \text{and} \qquad x^2.$$

Thus, classifiers in $\mathcal{F}_2$ have the form

$$\text{sign}(w_1 x + w_2 x^2 + b).$$

What is the smallest possible error of a classifier in $\mathcal{F}_2$? Give a classifier that achieves this error.

Exercise 4c (2). (*Errors of a constant classifier.*) Let

$$\mathcal{F}_0 = \{\text{always predict } +1\}.$$

What is the approximation error of $\mathcal{F}_0$? What is its estimation error? Explain your answer.

Exercise 4d (4). (*High-degree features with few samples.*) Let $\mathcal{F}_3$ be the set of linear classifiers that use the features

$$x, x^2, \ldots, x^{15}.$$

Compare $\mathcal{F}_2$ and $\mathcal{F}_3$ in terms of approximation error.

Now suppose you have only 10 training examples. Which class would you expect to have larger estimation error, $\mathcal{F}_2$ or $\mathcal{F}_3$? Explain why.

Which class would you choose in this situation? Describe how you could compare the two classes using the training data, without using the test set.

5 Bias-Variance Tradeoff in Linear Models (38 points)

To understand the bias-variance tradeoff, we can define and decompose the expected test error as:

$$\mathbb{E}_{x,y,\mathcal{D}}\left[(h_{\mathcal{D}}(x)-y)^2\right] = \underbrace{\mathbb{E}_{x,\mathcal{D}}\left[(h_{\mathcal{D}}(x)-\bar{h}(x))^2\right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{x}\left[(\bar{h}(x)-\bar{y}(x))^2\right]}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{x,y}\left[(\bar{y}(x)-y)^2\right]}_{\text{Noise}}$$

where $\mathcal{D}$ is the dataset composed of the feature x and the label y, $h_{\mathcal{D}}$ is a predictor learned from $\mathcal{D}$ by some machine learning algorithm $\mathcal{A}$, $\bar{h}$ is the expected predictor, and $\bar{y}$ is the expected label.

For simplicity, we only consider scalar features in this problem, but the insights can be generalized to feature vectors. Consider non-random scalar features $\{x^{(i)}\}_{i=1}^N$ where $x^{(i)} \neq x^{(j)}$ for all $i \neq j$. The labels are generated as $y^{(i)} = w^\star x^{(i)} + \epsilon^{(i)}$, where $w^\star$ is fixed and $\epsilon^{(i)}$ are i.i.d. noise terms sampled from $\mathcal{N}(0,\sigma^2)$. Note that $w^\star$ is unknown and $\epsilon^{(i)}$ is independent of $x^{(i)}$. The observed dataset is defined as $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$.

Ridge regression optimizes the following objective with $\lambda \geq 0$:

$$w_{\mathcal{D}} = \arg\min_w \frac{1}{N}\sum_{i=1}^{N}\left(wx^{(i)} - y^{(i)}\right)^2 + \lambda w^2.$$

For simplicity, the intercept term is omitted from this problem. We can see that with $\lambda = 0$, the objective becomes ordinary least squares regression, i.e., linear regression. The regularization term is the L_2 norm for the weight w.

Exercise 5a (5). (*Closed-form ridge solution.*) Show your step-by-step derivation of the closed-form solution of the ridge regression from the given objective.

Exercise 5b (4). (L_2 *regularization via artificial samples.*) Show that we can obtain L_2 regularization by adding artificial samples to the dataset and training with ordinary least squares regression. Describe your method for creating these artificial samples, particularly their features and labels.

Exercise 5c (3). (*Expected label and the noise term.*) Consider the expected label $\bar{y}(x) = \mathbb{E}_{y|x}[y]$. Show that $\bar{y}(x) = w^\star x$. Similarly, consider the noise term:

$$\text{Noise} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{y^{(i)}|x^{(i)}}\left[\left(\bar{y}(x^{(i)}) - y^{(i)}\right)^2\right].$$

Show that Noise $= \sigma^2$.

Exercise 5d (4). (*Expected ridge weight.*) Given a machine learning algorithm $\mathcal{A}$, we have $h_{\mathcal{D}} = \mathcal{A}(\mathcal{D})$. In our case, $h_{\mathcal{D}}(x) = w_{\mathcal{D}}x$. Consider the expected predictor $\bar{h} = \mathbb{E}_{\mathcal{D}\sim P^N}[h_{\mathcal{D}}]$; in our case, $\bar{w} = \mathbb{E}_{\mathcal{D}}[w_{\mathcal{D}}]$. Let $s^2 = \frac{1}{N}\sum_{i=1}^{N}\left(x^{(i)}\right)^2 > 0$. Show that:

$$\bar{w} = \frac{s^2}{\lambda + s^2}\, w^\star.$$

Exercise 5e (4). (*Squared bias of ridge regression.*) Consider the squared bias term:

$$\text{Bias}^2 = \frac{1}{N}\sum_{i=1}^{N}\left(\bar{w}x^{(i)} - \bar{y}(x^{(i)})\right)^2.$$

Show that:

$$\text{Bias}^2 = \left(\frac{\lambda}{\lambda + s^2}\right)^2 (w^\star)^2 s^2.$$

Exercise 5f (6). (*Variance of ridge regression.*) Consider the variance term:

$$\text{Variance} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{\mathcal{D}}\left[\left(w_{\mathcal{D}}x^{(i)} - \mathbb{E}_{\mathcal{D}}[w_{\mathcal{D}}x^{(i)}]\right)^2\right].$$

Show that:

$$\text{Variance} = \frac{s^4\sigma^2}{N(\lambda + s^2)^2}.$$

Exercise 5g (3). (*Limiting behavior as $\lambda \to 0$ and $\lambda \to \infty$.*) What happens to the Bias^2 and Variance terms when $\lambda \to 0$ and $\lambda \to \infty$? Your answer should demonstrate that the bias and variance are monotonic with respect to λ, but in different directions. Therefore, changing λ controls the tradeoff. In practice, since we do not know $w^\star$ and the true distribution of ϵ, we cannot infer the optimal value of λ. Therefore, we use model selection to determine the best value for λ.

Exercise 5h (3). (*Constrained form and a distance bound.*) Alternatively, we can consider an equivalent form of ridge regression:

$$w_{\mathcal{D}} = \arg\min_{w} \frac{1}{N}\sum_{i=1}^{N}\left(wx^{(i)} - y^{(i)}\right)^2 \quad \text{subject to} \quad w^2 \leq R.$$

The regularization constraint forces the weight w to be inside a ball around the origin with radius $\sqrt{R}$. Use the triangle inequality to show that:

$$|w_{\mathcal{D}} - \bar{w}|^2 \leq 4R.$$

From there, we can see that the maximum Euclidean distance between any two points in the ball is at most $2\sqrt{R}$.

Exercise 5i (3). (*A variance bound independent of $w^\star$ and ϵ.*) Show that ridge regression bounds the variance by $4Rs^2$:

$$\text{Variance} \leq 4Rs^2.$$

Note that this bound does not depend on $w^\star$ or ϵ, but it can be loose compared to the actual value of the variance.

Exercise 5j (3). (*Summarizing the bias-variance tradeoff.*) From the solutions to all questions in this problem, restate your understanding of the bias-variance tradeoff in linear models in your own words.

6 Programming (17 points)

To understand model selection using cross-validation, we will classify handwritten digits from the MNIST dataset. We will use the first 10 000 examples for training and the final 10 000 examples for testing. Do not modify the code below. Only add your code after it.

```
from sklearn.datasets import fetch_openml

print("Loading MNIST...")
X, y = fetch_openml("mnist_784", version=1, as_frame=False, return_X_y=True)
print(f"Loaded {len(X)} examples with {X.shape[1]} features.")
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]

X_train, y_train = (X_train[:10000], y_train[:10000])  # reduce dataset size
print(f"Using {len(X_train)} for cross-validation and {len(X_test)} for test.")
```

6.1 Warmup

Implement 5- or 10-fold cross-validation that tunes at least 3 estimators of your choice (for example, Logistic Regression, Naive Bayes, Decision Tree, etc.). For each estimator, select at least one hyperparameter and three candidate hyperparameter values for tuning. You may use any estimator provided by `sklearn`, but only `sklearn` estimators are allowed.

For each fold, train a new estimator and record the validation accuracy. The best estimator is the one with the best mean cross-validation accuracy. Using the best hyperparameter and corresponding estimator, refit on all 10 000 training examples and evaluate the best estimator on the test set. As you may have guessed, using the built-in `GridSearchCV` and similar classes is not allowed. Feel free to explore various hyperparameter search approaches, such as random search, etc.

6.2 Hand-Crafted Feature Engineering (4 points)

So far, we have used the raw pixel values of MNIST images as input features. However, the choice of feature transformation ϕ can significantly affect the performance of a classifier. In this task, you will explore whether manually designed features can improve classification accuracy.

Each MNIST image consists of 28×28 pixels, represented as a vector $x \in \mathbb{R}^{784}$. We can transform the original features into a new representation using a **feature mapping function**:

$$\phi : \mathbb{R}^{784} \to \mathbb{R}^d.$$

Part A: Designing Feature Transformations

Design and implement **at least two different feature transformations**, $\phi_1(x)$ and $\phi_2(x)$, that capture potentially useful characteristics of handwritten digits.

Below are several techniques you may consider.

1. **Average Pixel Intensity (Easy)**

   Different digits occupy different amounts of space in an image. For example, the digit 1 typically contains fewer foreground pixels than the digit 8.

   Compute the average pixel intensity:

   $$\phi_{\text{intensity}}(x) = \frac{1}{784}\sum_{i=1}^{784} x_i.$$

   This feature summarizes the overall amount of ink in the image, assuming larger pixel values represent foreground strokes.

2. **Horizontal and Vertical Symmetry**

   Some digits (e.g., 0 and 8) tend to be more symmetric than others (e.g., 2 and 7).

   One way to measure symmetry across the vertical axis is to compare the original image with its horizontally flipped version:

   $$\phi_{\text{symmetry}}(x) = -\frac{1}{784}\sum_{i=1}^{784} |x_i - x_i^{\text{flip}}|,$$

   where x^{flip} represents the horizontally flipped image. Larger values indicate greater symmetry.

   Extend this approach to measure symmetry across the horizontal axis.

3. **Spatial Pixel Distribution**

   Instead of computing the average intensity over the entire image, divide the image into several regions (e.g., a 4×4 grid).

   For each region, compute its average pixel intensity. This produces a feature vector representing **where the digit's strokes are concentrated**, rather than simply measuring their total intensity. This is a spatial extension of Technique 1.

Part B: Evaluation and Analysis

Using the best estimator from the previous task:

1. Compare the classification performance using:

   (a) The original 784 pixel features.

   (b) Your first feature transformation $\phi_1(x)$.

   (c) Your second feature transformation $\phi_2(x)$.

   You may concatenate your engineered features with the original features.

2. Use the same cross-validation folds to evaluate different representations. You may tune the estimator's hyperparameters separately for each representation.

3. Report the mean cross-validation accuracy for each representation and the dimensionality of the resulting feature vectors.
4. Explain what information each transformation captures and why you expect it to improve classification accuracy.
5. **Did the engineered features improve performance?** Explain your observations. Consider whether the additional features provide useful information or introduce redundancy and noise.

Important: You must implement your feature transformations yourself using basic numerical operations (e.g., NumPy). Calling existing feature extraction functions, such as `skimage.feature.hog()`, or using pretrained feature extractors is not allowed. The test set must not be used to design or select feature transformations. Use cross-validation for feature selection and model selection, and evaluate your final selected model on the test set only once.

Submission Requirements

Submit your completed implementation (1 file only) together with the assignment (or add it directly to the assignment) and report the following:

- All hyperparameter configurations. Documenting these directly in the code is also allowed.
- Each fold accuracy and the mean cross-validation accuracy for all estimators.
- The best hyperparameter, estimator, and the test accuracy.
- Decisions you made during the development process, for example, estimator selections, handling feature scaling, etc. You will surely need to make many decisions during the development process, so we would like to know them.

While we allow great flexibility in this problem, the cross-validation pipeline is relatively standard, so there is an overall minimal structure that we expect you to implement correctly. We will randomly select a subset of the submission to reproduce, so make sure your code reproduces your reported results. We do not ask for complete reproduction, but a high discrepancy will affect your grading.

Grading Rubric

You are expected to reach at least 96% accuracy on the test set. The top 10 highest-performing submissions, given reproducible results and a semantically correct implementation, will receive 10 bonus points for this homework. The total points for this homework are capped at 100.

Component	Points
Correct, concise, and reproducible implementation of the cross-validation pipelines (e.g., no test leakage,...)	5
Clear report of fold accuracy and mean cross-validation accuracy, best hyperparameter and estimator, and test accuracy	3
Detailed decisions and rationales behind the implementation	5
Good performance (passed minimum threshold)	4
Total	**17**